

Time Series Data Forecaster Agent

Technical Architecture & System Documentation

1. System Overview

The Time Series Data Forecaster Agent is a production-grade, multi-agent autonomous system designed to bridge the gap between raw temporal data and strategic business intelligence. Most traditional forecasting tools provide numerical predictions in a vacuum, lacking the context and justification required for high-stakes executive decision-making.

This system automates the entire forecasting lifecycle—from ingestion and data hygiene to statistical modeling and narrative synthesis. By orchestrating specialized agents that combine the mathematical rigor of `statsmodels` and `pmdarima` with the reasoning capabilities of Large Language Models (Gemini/Ollama), the system does more than just generate a forecast; it provides a theoretically grounded justification for why a specific model was chosen and what the results actually mean for the business.

Through a dedicated RAG (Retrieval-Augmented Generation) layer, the agent transforms raw statistical output into a comprehensive, executive-level strategic report, effectively functioning as an automated Senior Business Analyst.

2. System Setup & Installation

2.1 Prerequisites

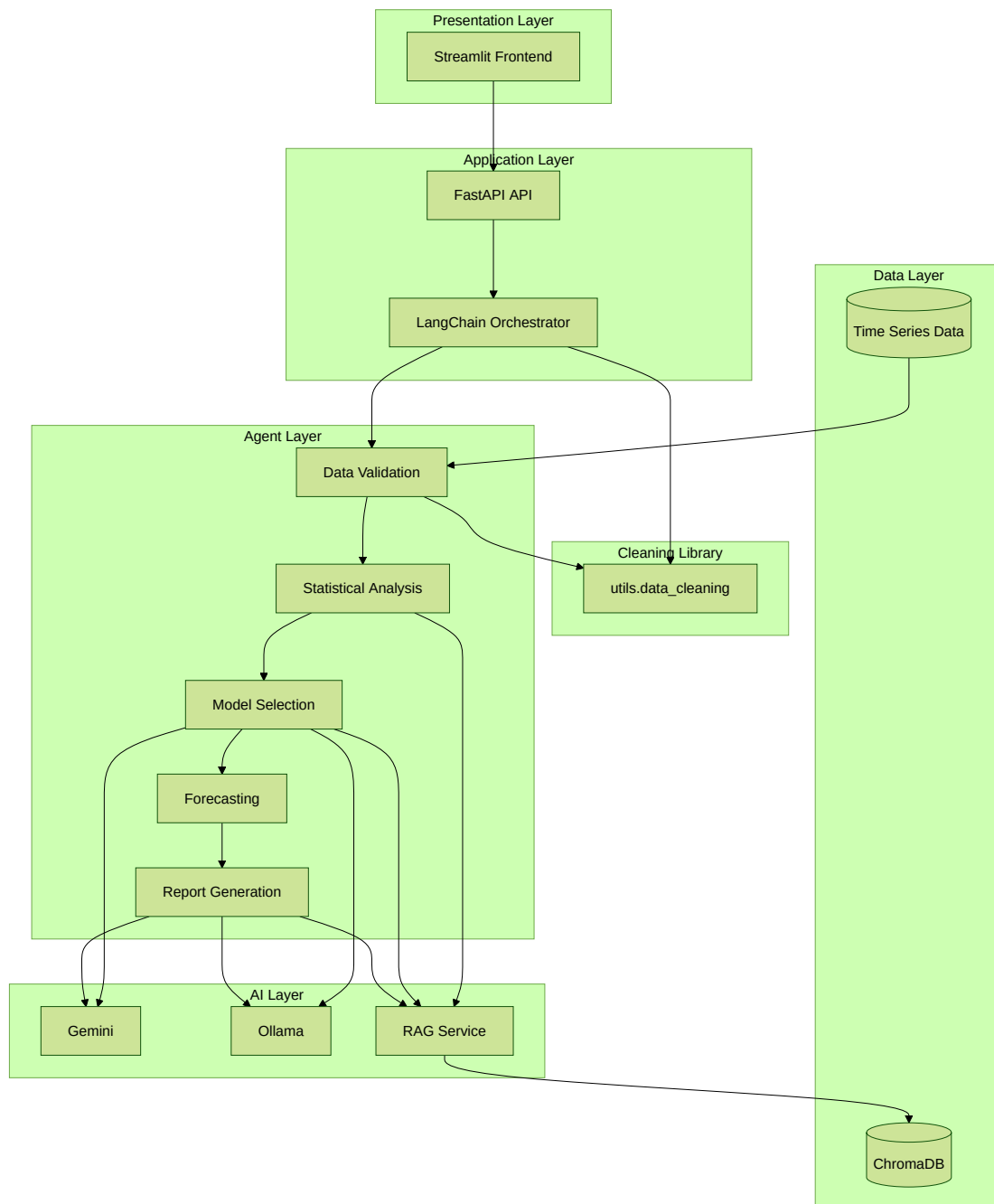
- Python 3.10+
- Docker & Docker Compose
- LLM API Key (Google Gemini) or local Ollama instance.

2.2 Local Environment Setup

```
git clone https://github.com/bmfmancini/  
data_forecasting_agent.git  
cd data_forecasting_agent  
pip install -r requirements.txt
```

Configure your `.env` file with your API keys and set your preferred LLM model (e.g., `gemini-pro`).

3. Architecture & Agent Roles



Agent Responsibilities

Agent	Primary Goal	Key Logic / Tools
Data Validation	Data Integrity	Reindexing, forward-fill / time interpolation / seasonal-decomposition imputation, duplicate

Agent	Primary Goal	Key Logic / Tools
		resolution (keep-first / mean / latest), IQR & Z-score outlier clipping, Isolation Forest multivariate outliers, EWMA & Savitzky–Golay smoothing, schema & sanity validation, Box-Cox stabilization.
Statistical Analysis	Numerical Fingerprinting	ADF/KPSS tests, STL decomposition, ACF/PACF lag analysis.
Model Selection	Decision Reasoning	Hybrid logic: Deterministic Suitability → LLM Reasoning → Heuristic Fallback.
Forecasting	Numerical Execution	Fitting via <code>pmdarima</code> and <code>statsmodels</code> , 95% CI generation.
Report Generation	Strategic Synthesis	RAG-enhanced narrative synthesis using an 8-section structure.

4. Operational Infrastructure & UX

4.1 Resource Management (Queueing System)

To prevent resource exhaustion from multiple simultaneous forecast requests, we implemented a Backend Queueing System. Incoming requests are processed sequentially, ensuring that each high-compute forecast has the required memory and CPU stability to complete without crashing the API.

4.2 Pre-Analysis & Demo Data

Users can load the Airline Passengers demo dataset to test the pipeline instantly. Before analysis begins, a Preflight Check button appears. This triggers a validation scan, allowing users to manually resolve issues or delegate the remediation to the AI.

4.3 Preflight Mathematics & Data Hygiene

During the Preflight Check, the Data Validation Agent executes a structured audit (mirroring the freeCodeCamp *How to Clean Time Series Data in Python* checklist) and applies a configurable set of mathematical transformations. All cleaning primitives are centralised in `backend/utils/data_cleaning.py`.

1. Time-Series Audit

Before any transformation runs, the agent calls `audit_series()` to produce a snapshot of the dataset's health: observation count, missing-value count, duplicate timestamps, regularity, inferred frequency and outlier counts (IQR + Z-score). The same snapshot is exposed to the LLM prompt and the UI so users can see exactly what was found.

2. Canonical Reindexing

Timestamps that are simply *missing from the index* (not represented as `NaN`) become explicit holes after `reindex_series()` aligns the data to a uniform frequency grid:

$$\text{index} = \{t_0, t_0 + \Delta, t_0 + 2\Delta, \dots, t_n\}$$

3. Linear Interpolation (Missing Data)

When minor sequential gaps are detected, the agent estimates the missing values by drawing a straight line between the known surrounding data points.

`impute_missing()` supports *forward-fill*, *time-weighted interpolation* and *seasonal-decomposition imputation*:

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0}$$

4. Seasonal Decomposition Imputation (Long Gaps)

For seasonal signals with long gaps, the series is decomposed into trend + seasonal + residual, and only the missing positions are reconstructed. This preserves the time-of-day or month-of-year pattern that pure interpolation would flatten:

$$\hat{y}_t = T_t + S_t \quad \text{where } T_t = \text{trend}, S_t = \text{seasonal component}$$

5. Duplicate-Timestamp Resolution

Pipeline retries frequently produce multiple readings for the same timestamp.

`resolve_duplicates()` collapses them via *keep-first*, *mean* or *latest*, preserving the invariant of a one-to-one index-to-value mapping.

6. IQR Outlier Clipping

Extreme anomalies that could skew the model are capped using the Interquartile Range (IQR) method. `apply_iqr_clipping()` /

`detect_outliers_iqr()` identify values outside the 1.5× IQR fence and either clip them to the boundary or replace them with `NaN` for downstream imputation:

$$IQR = Q_3 - Q_1$$

$$\text{Bounds} = [Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$$

7. Z-Score Outlier Clipping

For approximately Gaussian data, `apply_zscore_clipping()` /

`detect_outliers_zscore()` cap observations whose absolute Z-score exceeds a configurable threshold (default 3.0):

$$z_t = \frac{y_t - \mu}{\sigma}, \quad |z_t| > \tau \implies \text{clip to } \mu \pm \tau\sigma$$

8. Isolation Forest (Multivariate Outliers)

When multiple correlated sensors are present,

`detect_outliers_isolation_forest()` catches combinations of readings that look normal individually but anomalous together. Although the current UI is univariate, the helper is ready for future multi-channel ingestion.

9. Noise Smoothing

Noisy sensors can be smoothed before modelling. `smooth_series()` exposes two industry-standard algorithms:

- **Exponentially Weighted Moving Average (EWMA):** recent observations get exponentially more weight, ideal for non-stationary signals:

$$\hat{y}_t = \alpha y_{t-1} + (1 - \alpha) \hat{y}_{t-1}$$

- **Savitzky–Golay filter:** fits a low-order polynomial over a sliding window, preserving peak shapes that simple averages would flatten.

10. Box-Cox Transformation

If the data exhibits heteroscedasticity (changing variance over time), the agent applies a Box-Cox power transformation to stabilize the variance, making the series more normal:

$$y_i^{(\lambda)} = \begin{cases} \frac{y_i^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(y_i) & \text{if } \lambda = 0 \end{cases}$$

11. Schema & Sanity Validation

After cleaning, `validate_schema()` runs a final pass against a configurable contract (expected frequency, maximum missing-rate, value range, duplicate-free index, monotonic index). The result is appended to the LLM prompt and surfaced in the Data Quality tab so users can confirm post-cleaning state.

4.4 Core Preprocessing Engine (`utils.data_cleaning`)

All data hygiene primitives are centralized in a single, well-tested module: `backend/utils/data_cleaning.py` . By sharing this core engine across `preflight.py` , the Data Validation Agent, and the Statistical Analysis Agent, the system guarantees absolute consistency in how data is treated, whether during manual user previews or autonomous agent execution.

The engine is designed to execute transformations in a strict chronological sequence to prevent cascading data errors: the data is first audited, structurally aligned (reindexed) to expose hidden gaps, imputed to fix those gaps, scrubbed of outliers, and finally validated against a strict schema.

The public API surface is documented via Google-style docstrings and categorized by pipeline stage below:

Stage 1: Auditing & Validation

Function	Purpose
<code>audit_series</code>	Generates the initial health snapshot (length, missing values, duplicates, regularity, frequency, and IQR/Z-score outlier counts) before any transformations occur.
<code>validate_schema</code>	Performs the final contract check post-cleaning (frequency, missing rate, value range, duplicates, monotonic index) to authorize the data for forecasting.
<code>_infer_seasonal_period</code>	Maps the inferred frequency alias to a distinct seasonal period (e.g., 12, 4, 52) to inform downstream validation.

Stage 2: Structural Alignment & Imputation

Function	Purpose
<code>reindex_series</code>	Forces the index onto a canonical frequency grid, converting implicit missing timestamps into explicit

Function	Purpose
	NaN rows.
<code>resolve_duplicates</code>	Collapses multiple readings at the exact same timestamp via <i>keep-first</i> , <i>mean</i> , or <i>latest</i> aggregation.
<code>impute_missing</code>	Executes the mathematical imputation outlined in Section 4.3 (forward-fill, time-weighted linear interpolation, or seasonal-decomposition).

Stage 3: Outlier Remediation

Function	Purpose
<code>detect_outliers_iqr</code>	Flags values outside the 1.5× IQR fence; recommended for skewed distributions.
<code>detect_outliers_zscore</code>	Uses a rolling Z-score window (default 24) to flag local anomalies in approximately Gaussian data.
<code>detect_outliers_isolation_forest</code>	Detects multivariate anomalies across correlated channels (forward-compatible feature).
<code>apply_iqr_clipping</code> / <code>apply_zscore_clipping</code>	Physically caps the identified outliers to their respective mathematical boundaries in-place.
<code>treat_outliers</code>	The unified agent entry point for executing <i>clip</i> , <i>winsorize</i> , <i>remove</i> , and <i>zscore_clip</i> strategies.

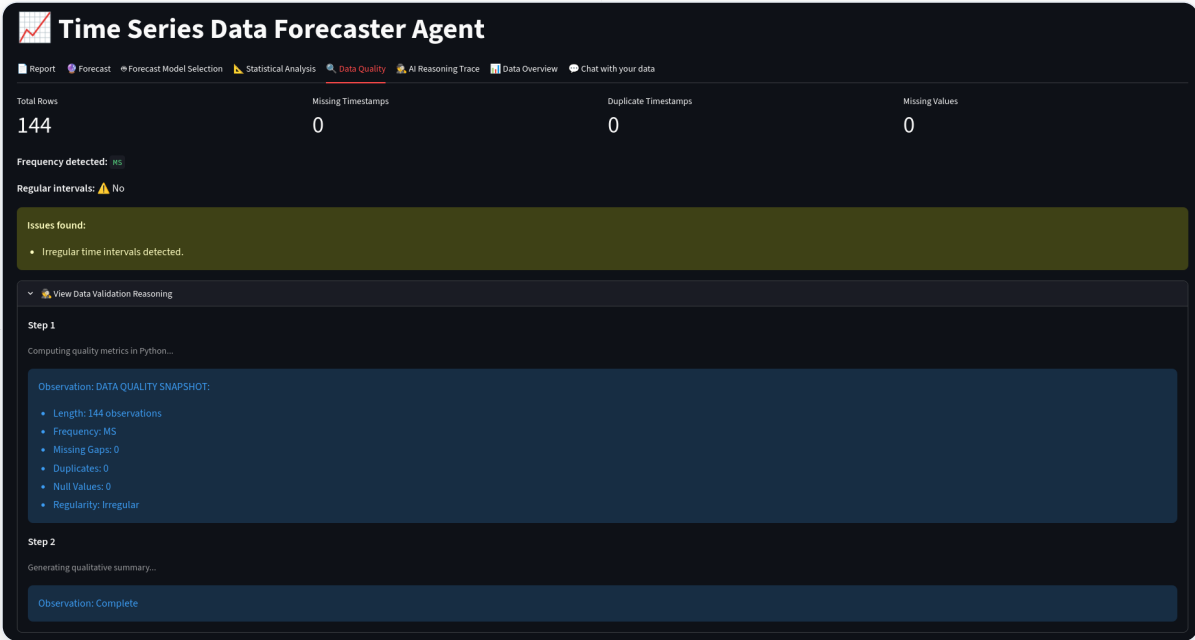
Stage 4: Noise Reduction

Function	Purpose
----------	---------

Function	Purpose
<code>smooth_series</code>	Applies EWMA or Savitzky–Golay filters for noise reduction prior to modeling.

4.5 Data Quality Tab (Business Analyst Persona)

The system features a dedicated Data Quality Tab. Here, the LLM adopts the persona of a Senior Business Analyst to provide a qualitative interpretation of the data's health, transforming raw counts into a bottom-line verdict on suitability for forecasting.



4.6 Forecast Model Selection Tab

In Auto mode, the system provides a detailed Rationale for why a specific model was chosen. This includes a model-by-model assessment, explaining why alternatives were rejected based on evidence like seasonality, trend, or data length.

is described as fast, interpretable, and robust for short-to-medium length series, matching the data characteristics implied by the evidence

Model-by-model assessment

- **ARIMA:** Detects the same seasonal period 12, but “plain ARIMA ignores seasonality, SARIMA may be better.” It would need differentiating to remove trend and seasonality, making it less directly aligned with the observed seasonal-trend structure.
- **SARIMA:** Confirms seasonality 12 and is “the gold standard for stationary-transformable seasonal series.” However, it “requires more data than ARIMA (at least 2 full seasonal cycles).” The evidence does not confirm that enough data are available, so the suitability is uncertain.
- **Holt-Winters:** Seasonality 12 is “detected – and Holt-Winters models seasonality natively.” Trend is present and Holt-Winters handles trend via exponential smoothing. The series is non-stationary, but Holt-Winters does not require pre-differentiating. Speed, interpretability, and robustness for short-to are explicitly highlighted.
- **EWMA:** Trend is detected, but “EWMA will lag behind trend changes,” making it less appropriate for a series with a clear upward slope. It is best for “short-term forecasts with stable patterns,” which does not align with the observed trend and seasonality.

Why alternatives were not selected

- ARIMA: Ignores the confirmed seasonality (period 12) and would need differencing to handle trend, making it less efficient for the given pattern.
- SARIMA: Although it models seasonality explicitly, the requirement of "at least 2 full seasonal cycles" is not verified by the evidence; without assurance of sufficient data, its practicality is uncertain.
- EWMA: Expected to "lag behind trend changes," which is undesirable given the identified upward trend; also does not natively address seasonality.

4.7 AI Reasoning Trace

For full auditability, the AI Reasoning Trace tab exposes the raw logs of agent interactions. Users can see the exact Python observations and thought process the LLM used to reach its conclusions.



Time Series Data Forecaster Agent

Report

Forecast

Forecast Model Selection

Statistical Analysis

Data Quality

AI Reasoning Trace

Data Overview

Chat with your data



Full AI Reasoning Trace

This log shows the internal 'monologue' and tool usage of the AI agents as they processed your request.

1. Data Validation Agent

Step 1

Computing quality metrics in Python...

Observation: DATA QUALITY SNAPSHOT:

- Length: 144 observations
- Frequency: MS
- Missing Gaps: 0
- Duplicates: 0
- Null Values: 0
- Regularity: Irregular

Step 2

Generating qualitative summary...

Observation: Complete

2. Statistical Analysis Agent

Step 1

Running ADF, KPSS, and STL in Python...

Observation: DOMAIN: User skipped or requested inference (AI must infer domain from stats) STATISTICAL PROFILE:

5. Model Selection Logic

The Model Selection Agent executes a multi-layered decision pipeline to ensure the most mathematically sound model is used.

5.1 The Decision Pipeline

1. **Deterministic Suitability (Python):** The agent first generates a Suitability Summary based on hard rules. It checks if Holt-Winters can handle the detected trend, if SARIMA has enough cycles (min 2), and if EWMA is too sensitive to the current outlier ratio.
2. **Cognitive Reasoning (LLM):** This summary is passed to the LLM, which weighs the pros and cons of each model to make a final selection based on the evidence.
3. **Heuristic Fallback (Safety):** If the LLM call fails or returns an invalid model, the system triggers a priority matrix (e.g., `if seasonal_period > 1 → SARIMA`) to ensure the pipeline never breaks.

5.2 Selection Modes

- Auto (AI Selects): Executes the full three-tier pipeline described above.
- Manual Selection: The user overrides the entire pipeline and forces the Forecasting Agent to fit a specific model.

5.3 Available Model Library & Core Mathematics

The Forecasting Agent utilizes the following mathematical frameworks to generate its predictions. The Model Selection Agent determines which of these is mathematically optimal for the current data signature.

1. ARIMA (Auto-Regressive Integrated Moving Average)

Best for non-seasonal, trended data. It models the next step as a linear function of past observations (AR) and past forecast errors (MA), applied to differenced data (I) to achieve stationarity.

$$y'_t = c + \phi_1 y'_{t-1} + \dots + \phi_p y'_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t$$

Where y'_t is the differenced series, ϕ represents AR parameters, and θ represents MA parameters.

2. SARIMA (Seasonal ARIMA)

Expands on ARIMA by adding seasonal backshifts. It is the primary model for datasets with confirmed recurring cycles (e.g., retail sales peaking every December).

$$\Phi_P(B^s)\phi_p(B)(1 - B^s)^D(1 - B)^dY_t = \Theta_Q(B^s)\theta_q(B)\epsilon_t$$

Where B is the backshift operator, and s is the seasonal period length.

3. Holt-Winters (Triple Exponential Smoothing)

Applies exponential weighting to level, trend, and seasonal components. It is highly effective for data where the trend and seasonal patterns are prominent but non-linear.

$$\text{Forecast: } \hat{Y}_{t+h} = L_t + hT_t + S_{t+h-s}$$

Where L_t is the level, T_t is the trend, and S_t is the seasonal component, each smoothed by constants α , β , and γ .

4. EWMA (Exponentially Weighted Moving Average)

A simpler smoothing technique that applies exponentially decreasing weights to older observations. Ideal for noisy, stable datasets without strong trends or seasonality.

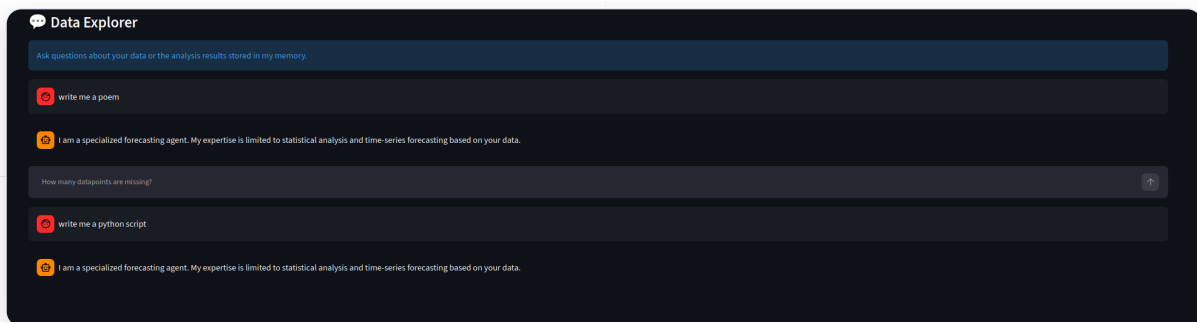
$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha)\hat{y}_t$$

Where α (between 0 and 1) is the smoothing factor dictating how heavily recent observations are weighted.

6. System Guardrails & Domain Constraint

To prevent prompt injection and ensure the agent remains a specialized tool, we implemented a strict Domain Guardrail layer.

The agent is explicitly instructed to only operate within the domain of Time Series Data Forecasting. If a user attempts to request general scripts or steer the conversation toward an unrelated domain, the agent will politely decline. This keeps the LLM's reasoning focused entirely on forecast accuracy.



7. Report Generation & Executive Synthesis

The Report Generation Agent converts raw numbers into a high-fidelity executive brief. Rather than a generic summary, it synthesizes several data streams into a strategic narrative.

7.1 Logic-Driven Visual Strategy

The agent determines which visualizations are necessary to communicate risk and insight to the C-suite:

- STL Decomposition: Triggered if strong seasonal patterns are detected.
- Confidence Intervals: Prioritized if the MAPE is high ($>15\%$), emphasizing risk and uncertainty.
- ACF/PACF Plots: Triggered specifically for SARIMA models to validate seasonal components.

7.2 AI Responsibility & Remediation

If the user chose "Let AI Decide" during preflight checks, the agent is instructed to assume full responsibility for those decisions. The report must explain the statistical rationale for the chosen remediation (e.g., linear interpolation) to ensure the user understands how the signal was preserved.

7.3 Report Structure

The final output is a strictly structured 8-section report covering strategic overview, historical trend analysis, future outlook, and tactical recommendations.

8. RAG Implementation & Knowledge Architecture

The Retrieval-Augmented Generation (RAG) component is integrated into the Report Generation Agent. Instead of providing a purely numerical output, the system leverages a domain-specific knowledge layer to transform raw forecasts into a theoretically grounded executive report.

8.1 The RAG Operational Workflow

1. **Embedding:** Domain documentation is processed through sentence-transformers , converting text into high-dimensional vectors.
2. **Storage:** These vectors are indexed in **ChromaDB** for persistent, fast retrieval.
3. **Retrieval:** The agent uses current forecast metrics as a query to pull the most relevant theoretical context from ChromaDB.
4. **Augmentation:** The LLM synthesizes the Numerical Forecast + Retrieved Knowledge → Contextual Executive Insight.

8.2 Knowledge Base Composition

The RAG knowledge base is a comprehensive library covering the full spectrum of time-series theory, including:

- Statistical Frameworks: Guides on ARIMA, SARIMA, and Holt-Winters.
- Stationarity & Testing: In-depth explanations of unit-root testing (ADF/KPSS) and differencing.
- Diagnostic Tools: Interpretations of ACF/PACF, STL decomposition, and periodograms.
- Validation & Metrics: Standardized definitions for RMSE, MAE, MAPE, and MASE.

8.3 Strategic Advantages

By utilizing local RAG, the system eliminates hallucinations by forcing the LLM to base its justifications on established statistical texts. This reduces the need

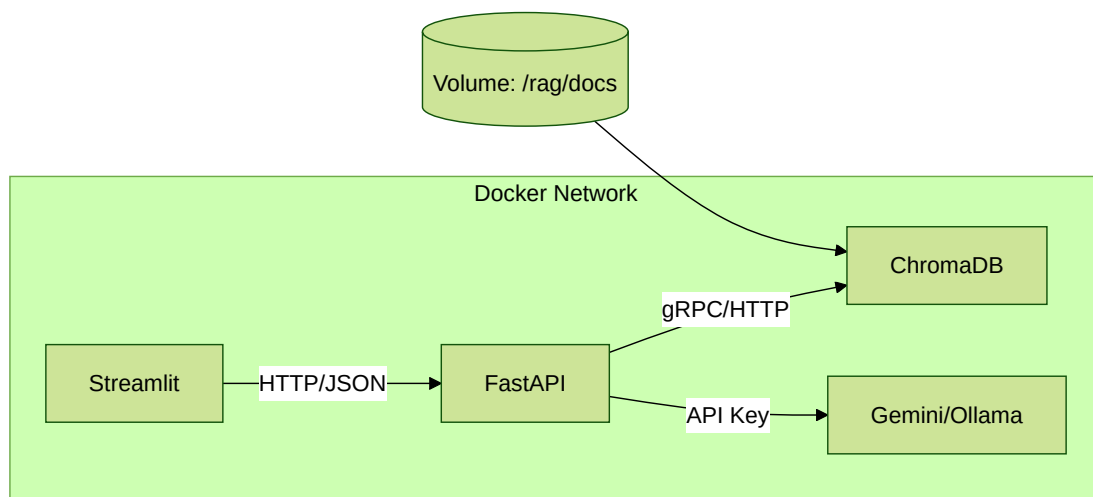
for iterative prompting, lowering both computational overhead and API costs.

9. Context-Aware Data Explorer

The Chat with Data interface allows users to query their dataset in natural language using a hybrid retrieval mechanism:

- Fact-Based Queries: Retrieves specific metrics (e.g., *"How many datapoints were missing?"*) from the stored Validation Summary in ChromaDB.
- Theoretical Queries: Queries the RAG Knowledge Base for forecasting theory (e.g., *"Why does stationarity matter here?"*).

10. Deployment Strategy



Deploy using `docker-compose up -d` to launch the Frontend, Backend API, and Vector Database as isolated containers.

11. Data Requirements & Constraints

To ensure mathematical accuracy and prevent pipeline failures, the input data must adhere to the following constraints:

- **Dimensionality:** The system currently strictly supports **Univariate Time Series** forecasting. The dataset must contain exactly one chronological time/date column and one numerical target/value column.
- **Data Volume & Model Suitability:** The minimum number of required data points is dynamic and depends entirely on the statistical model being evaluated:
 - *ARIMA / EWMA*: Requires a sufficient baseline of points to establish a trend (typically 30+ observations recommended).
 - *SARIMA / Holt-Winters*: Requires a minimum of **two full seasonal cycles** to detect and validate recurring seasonal patterns (e.g., at least 24 months of data for a monthly dataset with an annual seasonal period).
- **Missing Values:** The Data Validation Agent can automatically handle and impute missing values via three industry-standard strategies — *forward-fill* (step signals), *time-weighted linear interpolation* (continuous signals) and *seasonal-decomposition imputation* (long gaps in seasonal data). Datasets with massive sequential gaps that exceed the seasonal cycle length still fail the Preflight Check and require manual remediation.
- **Outliers:** Detected and treated using the IQR method, the rolling Z-score method, or the Isolation Forest algorithm for multivariate streams. Treatment options include clipping (Winsorize), removing and Z-score clipping. The agent's default when the user selects *Let AI Decide* is IQR Winsorize.
- **Duplicates:** Pipeline retries that produce multiple readings for the same timestamp are collapsed via *keep-first*, *mean* or *latest* aggregation.
- **Noise:** Optional smoothing via EWMA or Savitzky–Golay filtering is available before forecasting for high-frequency noisy sensors.

12. API Reference & Core Endpoints

The FastAPI backend uses an asynchronous, queue-based architecture to handle high-compute forecasting tasks without blocking the main thread or causing Out-Of-Memory (OOM) crashes. Here is the core API surface connecting the Streamlit frontend to the Langchain orchestrator:

Method	Endpoint	Primary Function
GET	/health	Basic health check to ensure the API is responsive.
POST	/upload	Accepts file uploads, validates MIME types and sizes, parses the data into a Pandas DataFrame, and stores it in ephemeral memory. Returns a unique <code>file_id</code> and detected metadata (date column, frequency).
POST	/preflight	Runs deterministic data quality checks on the uploaded dataset (using <code>file_id</code>) before initiating the full pipeline. Allows users to catch formatting issues early.
POST	/analyze	Submits the data for full forecasting analysis. Places the request into a FIFO queue (<code>JOB_QUEUE</code>) and returns a <code>job_id</code> .
GET	/jobs/{job_id}	Polling endpoint to retrieve the real-time status, execution progress (percentage), current step, and final results of a forecasting job.
POST	/chat	Context-aware data explorer endpoint. Uses the <code>file_id</code> to query both the raw DataFrame and the RAG-indexed analysis results (ChromaDB) for natural language Q&A.

Resource Management Note: To prevent OOM errors, the API enforces strict in-memory storage limits, automatically evicting the oldest entries when limits are reached (Max Files: 50, Max Jobs: 100).

13. Data Privacy & Security

Given the sensitive nature of corporate forecasting data, the system is designed with strict boundaries around data persistence and LLM communication:

- **Ephemeral Data Storage:** Uploaded datasets (CSVs) are stored strictly in-memory within the backend container. They are never written to disk as raw files. If the container restarts, or if the 50-file limit is reached, the data is permanently wiped from memory.
- **Vectorized Abstraction:** Only the *analysis results* and *metadata* are passed to ChromaDB for persistent RAG interactions. The raw transactional data remains isolated.
- **LLM Routing & Privacy:**
 - **Local Mode (Ollama):** For air-gapped or highly sensitive environments, all data and reasoning traces remain entirely on-premises within the Docker network.
 - **Cloud Mode (Google Gemini):** If using the cloud LLM, only the statistical profile (JSON) and synthesized context are sent via the API.
- **CORS Restrictions:** The API explicitly restricts Cross-Origin Resource Sharing (CORS) strictly to the designated frontend containers (e.g., `http://localhost:8501` , `http://frontend:8501`).

14. Limitations & Challenges

14.1 Technical Challenges

- Resource Spikes: Solved via the Backend Request Queue.
- Trust/Auditability: Solved by the Reasoning Trace, Business Analyst Persona, Model Selection Rationale and the structured `validate_schema()` post-cleaning report.
- LLM Hallucinations: Mitigated by constrained prompting and RAG grounding.
- Data Quality Drift: Solved by the centralised `utils.data_cleaning` library which provides a single, well-tested set of cleaning primitives (audit, reindex, impute, treat outliers, smooth, validate) used by both the preflight pipeline and the agents.

14.2 Current Limitations

- Limited to classical statistical models (No LSTM/Prophet integration).
- Designed for batch uploads rather than real-time data streams.
- UI exposes a curated subset of cleaning options (forward-fill, interpolation, IQR / Z-score clipping); advanced options such as *seasonal-decomposition imputation*, *EWMA smoothing*, *Savitzky–Golay filtering* and *Isolation Forest* are available programmatically and will be added to the UI in a follow-up release.

15. Acknowledgments & Attributions

This project was made possible by the foundational work, research, and open-source contributions of the following individuals and organizations:

- [Bala Priya C](#) for the data cleaning methods in the freeCodeCamp post [How to Clean Time Series Data in Python](#) that inspired the `utils.data_cleaning` module.
- [Forecasting: Principles and Practice \(3rd ed.\)](#) by Hyndman & Athanasopoulos — the canonical reference for the ARIMA, SARIMA, and exponential-smoothing methodology implemented in the `backend/forecasting` package.
- [Diogo Franquinho](#) for his excellent Udemmy courses and concise *Time Series Analysis* technical notes covering stationarity, ACF, ARIMA, and AIC-based model selection that deeply informed the RAG knowledge base (`backend/rag/docs/`).

Open-Source Ecosystem

- [Statsmodels](#) for rigorous statistical modeling.
- [Pmdarima](#) for automated ARIMA modeling.
- [LangChain](#) for LLM orchestration and agent integration.
- [Streamlit](#) for the responsive frontend framework.
- [FastAPI](#) for the high-performance backend framework.